LABORATORY MANUAL · WEEK 5

Lab 05 — Builds, Dependencies and Versioned Artifacts

DevOps Principles and Practices (2+1) · BS Computer Science · National Textile University

Duration	3 hours (one laboratory session)
Mapped outcome	L-CLO-1 — Construct an automated continuous integration pipeline using industry-standard tools. (P3 — Precision, Psychomotor)
Graduate attribute	GA-5 — Modern Tool Usage; GA-3 for the versioning and packaging decisions
Prerequisite	Lab 02 complete (a working Makefile with `check`, `build`, `test`, `clean`, `ci`) and Lab 04 complete (your team repository on GitHub)
Environment	Ubuntu 24.04 under WSL2 (or native Linux / macOS Terminal). Node.js 20 or newer and npm.
Deliverable	One `.tgz` artifact, one repository URL and a short report — see Section 9
Weight	Counts towards the 35 % weekly laboratory task component

Install Node.js before the session. Inside WSL2: `sudo apt update && sudo apt install -y nodejs npm`, then check with `node -v` and `npm -v`. If `node -v` reports anything below v20, use the NodeSource instructions in `scripts/README.txt` instead. Students who install Node during lab time lose twenty minutes of a three-hour session.

1. Why this lab follows Lab 02 and Lab 04

In Lab 02 you wrote a Makefile that knew the **order** of the work and the **recipe** for it. The recipe was one line of `tr`. That was fine for uppercasing text files, and it is exactly how real build systems are not written.

Today the recipe moves out. Make keeps the job it is good at — working out what is out of date and in what order — and a real build tool takes over the job of actually producing the output, including the job Make cannot do at all: **fetching the code you depend on**.

Four things happen today:

1. You declare a dependency, and then look at the **seven** packages that arrive because of it. The gap between what you asked for and what you got is the whole subject of this lab.
2. You replace the recipe inside the Makefile's build rule with the build tool. `make ci` still works — Week 06 is going to hand that exact target to a build server.
3. You use a **lock file** to make the install reproducible, and prove it rather than assume it.
4. You produce a **versioned artifact**, check that it actually works, and publish it to a repository.

A note on what is being assessed. Task 4 is the marked one — the artifact and whether it runs for somebody who is not you. Tasks 1 to 3 build up to it. Task 5 is compulsory and carries rubric marks through the URL you submit.

The outline for this course names **Maven, Gradle or npm**. This manual uses **npm**, because the lecture's worked figures were npm and because every one of you already has JavaScript on your machine. The concepts transfer unchanged: `pom.xml` and `build.gradle` are `package.json`, Maven Central and the Gradle cache are the registry, and `mvn package` is `npm pack`.

2. Objectives

By the end of this session you will be able to:

- Declare a direct dependency and explain the difference between it and the transitive dependencies it drags in.
- Read `npm ls --all` and point at a dependency that was **deduped**, and say what would have happened without that.
- Explain what `^15.0.3` permits and what it forbids, and why that is a reproducibility problem.
- Say precisely what a lock file records that `package.json` does not, and when `npm ci` refuses to run.

- Drive a build tool from a Makefile without the two of them fighting over the same work.
- Choose the next version number for a change and defend the choice using semantic versioning.
- Produce a versioned artifact, inspect what is inside it, and verify it works as a consumer would receive it.
- Publish that artifact to a repository, and explain why a published version can never be changed.

3. Before you start

3.1 Check your toolchain

```
node -v
npm -v
make --version | head -1
git --version
```

Node 20 or newer. If `npm -v` fails but `node -v` works, install npm separately — on Ubuntu they are two packages.

3.2 Set up the working copy

Start from your own Lab 02 work:

```
mkdir -p ~/devops/lab05
cp -r ~/devops/lab02/Makefile ~/devops/lab02/src ~/devops/lab05/
cd ~/devops/lab05
make ci
```

It must pass before you change anything. A build you cannot run is not a baseline.

> **If you no longer have your Lab 02 work.** The provided `fallback/` folder holds the Lab 02 model Makefile and its three `src/*.txt` files. Copy those in instead — and **say so in your report**. You lose no marks for using it; you lose marks for not saying.

3.3 Take the provided files

Unzip `Lab05-student-files.zip` into `~/devops/lab05`. You are given three things and you will type everything else:

Path	What it is
`src/*.md`	Three Markdown sources, replacing the three `.txt` files from Lab 02. Data, not code.
`tools/build.js`	The build program itself — about twenty lines of Node. This is the *application*, not the subject of the lab, so it is given to you. Read it; you need to know what it expects on the command line.
`test/build.test.js`	Two tests, written against Node's own test runner. Task 2 asks you to add a third.
`fallback/`	The Lab 02 model answer, for Section 3.2 only.

Read `tools/build.js` now. Two lines matter for everything that follows:

```
const MarkdownIt = require('markdown-it');      // <- the dependency
const [src, dest] = process.argv.slice(2);      // <- build.js SOURCE DEST
```

The first line is why you need a package manager at all: that module is not in Node and not in your repository. The second is the contract the Makefile has to satisfy.

1 Declare a dependency, then look at what arrived — 25 minutes

Not assessed directly. Everything else depends on reading this output correctly.

4.1 Create the manifest

```
cd ~/devops/lab05
npm init -y
cat package.json
```

`npm init -y` writes a `package.json` with defaults and asks you nothing. It is the manifest — the file that says what this project is called, what version it is, and what it needs. Open it and set three fields by hand:

```
"name": "devops-pipeline",
"version": "1.0.0",
"description": "Renders the course Markdown sources into HTML pages.",
```

> **The name has rules and the version has rules.** A package name must be lowercase, with no spaces. The version must be three numbers separated by dots — `1.0.0`, never `1.0` and never `v1.0.0`. npm refuses anything else, and Task 4 depends on it being a valid semantic version.

4.2 Declare one dependency

```
npm install markdown-it
```

Expected output

```
added 7 packages, and audited 8 packages in 2s

4 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Read the first line again. You asked for **one** package and npm installed **seven**. Nobody told you about the other six, and you are now responsible for all of them.

> **Your numbers may differ from the ones printed here.** Package versions move. If your output says six packages or nine, that is not an error — record **your** number in your report, not the one in this manual. The point is the ratio, not the integer.

4.3 Three places the dependency is now recorded

```
cat package.json | grep -A3 dependencies
ls node_modules | head
wc -c package.json package-lock.json
```

Three artefacts, three different jobs:

File	What it holds
`package.json`	What you **asked for**: `"markdown-it": "^15.0.3"`. One line. A *range*, not a version.
`node_modules/`	The actual code, on disk, ready to run. Never committed to Git — it is an output, not a source.
`package-lock.json`	What you **got**: every one of the seven packages, its exact version, and a checksum. Always committed.

The size difference is the lesson. On the run this manual was verified against, the lock file was **seven times larger** than the manifest, for one declared dependency — and the lecture quoted 288 bytes against 29,702 for a single `express`. Your numbers will differ; what matters is the ratio:

Expected output

```
     511 package.json
    3607 package-lock.json
```

4.4 Find the transitive tree, and the diamond in it

```
npm ls --all
```

Expected output

```
devops-pipeline@1.0.0 /home/you/devops/lab05
`-- markdown-it@15.0.3
  +-- argparse@3.0.3
  +-- entities@8.1.0
  +-- linkify-it@6.1.0
  | `-- uc.micro@3.0.0 deduped
  +-- mdurl@2.1.0
  +-- punycode.js@2.3.1
  `-- uc.micro@3.0.0
```

One direct dependency, six transitive ones. And look at `uc.micro`: it appears **twice**, because `markdown-it` needs it and `linkify-it` needs it too. The second one says **deduped**.

> **That word is the lecture's diamond, in your own output.** Two packages wanted `uc.micro`. Their ranges overlapped, so npm installed **one** copy and pointed both at it. Had the ranges not overlapped — one wanting 2.x and one wanting 3.x — npm would have installed both, nested, and your program would be running two different versions of the same library at once. Find the `deduped` line in your own tree before moving on.

4.5 What the range actually permits

Your `package.json` says `^15.0.3`. Test what that means instead of guessing:

```
npx semver -r '^15.0.3' 15.0.3 15.9.9 16.0.0
```

Expected output

```
15.0.3
15.9.9
```

`16.0.0` is missing from the output because the caret permits anything up to, but not including, the next major version. In other words `>=15.0.3 <16.0.0`. Your manifest does not name a version. It names a **set** of versions, and which member of that set you get depends on what day you install.

4.6 Checkpoint

- You can state how many packages you asked for and how many you have.
- You can point at a `deduped` line in your own `npm ls --all` output and say what it means.
- You can say what `^15.0.3` permits and what it forbids.

2 Move the recipe out of the Makefile — 30 minutes

Not assessed directly.

The Makefile currently does the work itself. After this task it will decide *what* to build and delegate *how* to the build tool. Nothing about `make ci` changes from the outside, which is the point — Week 06 hands that target to a build server and must not need to know what is behind it.

5.1 Switch the sources over

Delete the three `.txt` files you copied in and keep the three provided `.md` files:

```
rm -f src/*.txt
ls src/
```

Expected output

```
greeting.md  notes.md  services.md
```

5.2 Give npm two scripts to run

In `package.json`, replace the placeholder `"scripts"` block with:

```
"main": "tools/build.js",
"bin": { "devops-build": "tools/build.js" },
"files": ["tools/", "README.md"],
"scripts": {
  "build": "node tools/build.js",
  "test": "node --test"
},
```

Check both by hand before you let Make near them:

```
mkdir -p out
npm run build -- src/notes.md out/notes.html
cat out/notes.html
```

Expected output

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<title>notes</title>
<h1>Notes</h1>
<ul>
<li>Make rebuilds only what changed</li>
<li>Timestamps decide, not contents</li>
<li>A build tool owns the <em>recipe</em>; Make owns the <em>order</em></li>
</ul>
</html>
```

> **The bare double dash is not decoration.** `npm run build -- src/notes.md out/notes.html` passes the two paths through to `node tools/build.js`. Leave the `--` out and npm swallows them as its own arguments, `build.js` receives nothing, and you get the usage message. This trips people up every time and the error does not explain itself.

5.3 Rewrite the build rule

Four edits to your Lab 02 Makefile. The structure is unchanged — only the recipe and the file extensions move. Every recipe line below begins with a **tab**, not spaces; if you paste from a PDF you will get spaces and Make will tell you so:

```
SOURCES := $(wildcard $(SRC_DIR)/*.md)
OUTPUTS := $(patsubst $(SRC_DIR)/%.md,$(OUT_DIR)/%.html,$(SOURCES))

node_modules: package-lock.json
	npm ci
	@touch node_modules
```

```
deps: node_modules  ## install exactly what the lock file says

$(OUT_DIR)/%.html: $(SRC_DIR)/%.md package-lock.json | $(OUT_DIR) node_modules
     @echo "  BUILD $< -> $@"
     @npm run --silent build -- $< $@

test: build  ## run the test suite against what was built
     @npm test --silent
```

Also change the `check` target to look for `*.md` instead of `*.txt`, and add `deps` to `.PHONY` — but **not** `node_modules`, because that one is a real directory and Make must be allowed to compare its timestamp.

Three decisions in that rule are worth more than the typing:

- `node_modules` is a **real target** with `package-lock.json` as its prerequisite, so changing the lock file reinstalls — automatically, exactly once, and only when needed. The `@touch` is there because `npm ci` does not reliably update the directory's own timestamp.
- `node_modules` is an **order-only** prerequisite of the pages, after the `|`. The pages must not be rebuilt merely because a package was reinstalled.
- `package-lock.json` is a **normal** prerequisite of the pages. A different dependency version is a different build, so every page is out of date. This is the reproducibility argument from the lecture, expressed as a dependency edge.

5.4 Prove the delegation works

```
make clean
make ci
```

Expect the three BUILD lines, the tests, and `npm pack` once you have added the `package` target in Task 4. `npm ci` runs **only** if `node_modules` is missing or older than the lock file — which is the whole reason it was written as a target rather than a command in the recipe. Now run it again:

```
make build
```

Expected output

```
make: Nothing to be done for 'build'.
```

Make still does incremental builds, even though it no longer knows how the pages are made. Now touch one dependency edge and watch it cascade:

```
echo '- A fourth note' >> src/notes.md
make build
```

One page rebuilds. Then edit the lock file's dependency edge instead:

```
npm install markdown-it@15.0.3
make build
```

All three rebuild, because `package-lock.json` changed.

> **If nothing rebuilds, look at the clock before you look at the Makefile.** Make compares timestamps at one-second resolution. If the file you changed and the output you expected to go stale were written inside the same second, Make will see them as equally old and do nothing. Wait a second and try again. This is a genuine property of Make, not a mistake in your rule — it was hit while writing this manual.

5.5 Add the third test

Open `test/build.test.js`. Two tests are written. Add a third that fails if a page is empty — a build that produces a zero-byte file is a build that has silently failed, and neither existing test catches it. Then:

```
make test
```

5.6 Checkpoint

- `make ci` passes from a clean tree, and the Makefile contains no `tr`, no `markdown`, and no HTML.
- A second `make build` says there is nothing to do.
- Editing `src/notes.md` rebuilds one page; changing `package-lock.json` rebuilds all three.
- Your third test exists and passes.

3 Lock files, and proving reproducibility — 30 minutes

Not assessed directly — but two rubric lines in Task 4 depend on understanding it.

6.1 Look inside the lock file

```
node -e "const l=require('./package-lock.json');
  for (const [k,v] of Object.entries(l.packages)) if (k) console.log(k, v.version);"
```

Expected output

```
node_modules/argparse 3.0.3
node_modules/entities 8.1.0
node_modules/linkify-it 6.1.0
node_modules/markdown-it 15.0.3
node_modules/mdurl 2.1.0
node_modules/punycode.js 2.3.1
node_modules/uc.micro 3.0.0
```

Seven entries, each with **one** exact version — no ranges anywhere. Now look at what else each entry carries:

```
grep -m1 -A4 '"node_modules/markdown-it"' package-lock.json
```

There is an `integrity` field holding a `sha512-` hash of the tarball. That is not for your benefit, it is for npm's: on install it hashes what the registry sent and refuses it if the hash does not match. A lock file does not only pin versions, it pins **contents**.

Records	`package.json`	`package-lock.json`
Your direct dependencies	yes	yes
Transitive dependencies	no	yes, all of them
Exact versions	no — ranges	yes
Content checksums	no	yes
Where each package came from	no	yes

6.2 npm install and npm ci are not the same command

Command	What it does
`npm install`	Reads `package.json`, resolves the ranges against the registry **today**, installs the result, and **rewrites** the lock file to match. For a developer adding a dependency.
`npm ci`	Ignores the ranges. Deletes `node_modules`, installs exactly what the lock file says, and **never** writes to the lock file. For a build server, and for the Makefile rule you just wrote.

`ci` stands for clean install, not continuous integration — though that is where it belongs.

6.3 Prove the install is reproducible

```
rm -rf node_modules && npm ci >/dev/null
find node_modules -type f | sort | xargs shasum | shasum > /tmp/h1
rm -rf node_modules && npm ci >/dev/null
find node_modules -type f | sort | xargs shasum | shasum > /tmp/h2
cmp /tmp/h1 /tmp/h2 && echo IDENTICAL
```

Expected output

```
IDENTICAL
```

Two installs, same bytes, down to a single hash over every file in the tree. Paste your hash into your report. That is what reproducible means, and it is the reason the lock file is committed.

6.4 Make the manifest and the lock file disagree

Deliberately break the invariant, which is the fastest way to learn what maintains it. Add a dependency to `package.json` **by hand**, without installing it:

```
"dependencies": {
  "markdown-it": "^15.0.3",
  "js-yaml": "^4.1.0"
}
```

```
npm ci
```

Expected output

```
npm error code EUSAGE
npm error
npm error `npm ci` can only install packages when your package.json and
npm error package-lock.json or npm-shrinkwrap.json are in sync. Please update
npm error your lock file with `npm install` before continuing.
npm error
npm error Missing: js-yaml@4.3.2 from lock file
npm error Missing: argparse@2.0.1 from lock file
```

> **This refusal is the single most useful thing npm does.** It is what stops a build server quietly installing something the lock file never approved. Note that it names **two** missing packages when you added one — `argparse` is `js-yaml`'s transitive dependency, and the lock file has to approve that too. Now undo the edit, remove `js-yaml` from `package.json`, and confirm `npm ci` is happy again.

6.5 Check the tree for known vulnerabilities

```
npm audit
```

Probably `found 0 vulnerabilities`, because this tree is small and current. What matters is that the question is answerable at all, and that it is answerable **because** the lock file names exact versions. A range cannot be audited: `^15.0.3` is not a thing that has or does not have a vulnerability. Week 16 scans container images the same way.

6.6 Checkpoint

- You can name two things the lock file records that `package.json` does not.
- You have two identical hashes from 6.3, and they are in your report.
- You have seen `npm ci` refuse to run, and can say why that refusal is a feature.

4 ASSESSED — produce a versioned artifact that works 45 minutes

> **This task is marked. Section 10 is the rubric.** The marks are not for running `npm pack`. They are for the artifact being correct — which is decided by whether it runs for somebody who does not have your `node_modules` directory.

7.1 Choose the version number, then set it

You have changed the build from `tr` to a Markdown renderer. The output format changed from `.txt` to `.html`. Decide what that is, using the lecture's rules, **before** you type anything:

Change	Means
PATCH — `1.0.1`	A bug fix. Nobody who uses it has to do anything.
MINOR — `1.1.0`	New capability, nothing existing broken. Safe to take.
MAJOR — `2.0.0`	Something that used to work no longer does. Consumers must act.

Write down which one you chose and the one-sentence reason, then let npm do the edit:

```
git add -A && git commit -m "Build with npm and markdown-it"
npm version minor
```

Expected output

```
v1.1.0
```

```
git log --oneline -1
git tag
```

Expected output

```
1ebf63d 1.1.0
v1.1.0
```

> **npm version did three things, not one.** It edited `package.json`, it made a commit, and it created an annotated Git tag. That is deliberate: a version number that exists in the manifest but not in the history is a version you cannot ever check out again. This is the Lab 03 tagging discipline, automated. It needs a clean working tree, which is why you commit first.

7.2 Decide what goes in the artifact

An artifact is not a copy of your repository. `files` in `package.json` lists what ships:

```
"files": ["tools/", "README.md"],
```

Write a short `README.md` — the artifact needs one, and npm includes it automatically. Then ask npm what it would ship, without shipping it:

```
npm pack --dry-run
```

Expected output

```
npm notice package: devops-pipeline@1.1.0
npm notice Tarball Contents
npm notice 119B README.md
npm notice 531B package.json
npm notice 761B tools/build.js
npm notice Tarball Details
npm notice name: devops-pipeline
npm notice version: 1.1.0
npm notice filename: devops-pipeline-1.1.0.tgz
npm notice total files: 3
```

Check that list against what you expect, item by item. Three files. Not `src/`, not `out/`, not `test/`, not `node_modules/`, not `Makefile`. If `node_modules` or `out/` appears in your list, your `files` field is wrong — fix it now, because an artifact that carries its own build outputs is how a stale build reaches production. `package.json` is always included whether you list it or not.

7.3 Build the artifact

```
npm pack
ls *.tgz
tar -tzf devops-pipeline-1.1.0.tgz
```

Expected output

```
package/tools/build.js
package/README.md
package/package.json
```

The filename is `name-version.tgz` and npm chose it, not you. That is the whole point of the lecture's "release identification" slide: the artifact carries its own identity, so a file on a disk six months from now can still be traced back to a commit — through the version, to the tag, to the commit.

Add a `package` target to the Makefile so the artifact is part of the pipeline rather than something you remember to do:

```
package: test  ## produce the versioned artifact
     npm pack

clean:  ## remove all generated files
     rm -rf $(OUT_DIR) *.tgz

ci: clean deps check test package  ## the build server entry point
```

Note what the package target depends on. `test`, which depends on `build`. It is not possible to produce an artifact from code whose tests have not passed, because Make will not let you. That is the build-once-deploy-many rule from the lecture, enforced by the dependency graph instead of by discipline.

7.4 Verify the artifact the way a consumer receives it

This is the part that is actually marked. Your artifact works on your machine; that proves nothing, because your machine has `node_modules`. Install it somewhere clean:

```
mkdir -p /tmp/consumer && cd /tmp/consumer
npm init -y >/dev/null
npm install ~/devops/lab05/devops-pipeline-1.1.0.tgz
printf '# From a consumer\n\nInstalled as a **published artifact**.\n' > in.md
npx devops-build in.md out.html
cat out.html
```

Expected output

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<title>in</title>
<h1>From a consumer</h1>
<p>Installed as a <strong>published artifact</strong>.</p>
</html>
```

Now look at what arrived with it:

```
npm ls --all
```

Expected output

```
consumer@1.0.0 /tmp/consumer
`-- devops-pipeline@1.1.0
  `-- markdown-it@15.0.3
    +-- argparse@3.0.3
    +-- entities@8.1.0
    +-- linkify-it@6.1.0
    | `-- uc.micro@3.0.0 deduped
    +-- mdurl@2.1.0
    +-- punycode.js@2.3.1
    `-- uc.micro@3.0.0
```

Your artifact is 962 bytes and contains three files, yet the consumer ended up with eight packages. The artifact did not carry its dependencies — it **declared** them, and npm resolved them again at the far end. Transitive dependencies are not a thing that happens to you once at install time; they are a thing your artifact does to everybody who uses it.

7.5 The mistake this task exists to catch

Go back to `~/devops/lab05` and move the dependency to the wrong place — `devDependencies` instead of `dependencies`:

```
"devDependencies": { "markdown-it": "^15.0.3" }
```

```
npm pack
cd /tmp && rm -rf bad && mkdir bad && cd bad && npm init -y >/dev/null
npm install ~/devops/lab05/devops-pipeline-1.1.0.tgz
printf '# x\n' > in.md
npx devops-build in.md out.html
```

Expected output

```
Error: Cannot find module 'markdown-it'
```

```
npm ls --all
```

Expected output

```
bad@1.0.0 /tmp/bad
`-- devops-pipeline@1.1.0
```

> **Read the second output, not the first.** The error says a module is missing. The tree says **why**: `devops-pipeline` has nothing underneath it. A `devDependency` is for building the package, not for running it, so npm correctly did not install it for the consumer — and your artifact is broken for everyone except you. It packs cleanly, it passes your tests, and it fails on first use. Now put it back under `dependencies`, repack, and confirm 7.4 works again.

This is the lab in one experiment: the thing that broke was not in your code, your tests or your build. It was in a declaration about *somebody else's* environment.

7.6 Checkpoint — this is what is marked

- `make ci` produces `devops-pipeline-<version>.tgz` from a clean tree.
- `tar -tzf` on it lists only the files you intended, and `node_modules` and `out/` are not among them.
- The artifact installs into an empty directory and `npx devops-build` runs there.
- A Git tag exists matching the version in `package.json`.
- You can explain the `devDependencies` failure from the tree output, not just from the error message.

5 Publish the artifact to a repository — 35 minutes

Compulsory, and it carries rubric marks through the URL you submit. Work in your **Lab 04 team repository** — this is the first thing the team repository is actually used for.

A `.tgz` on your laptop is not an artifact repository. An artifact repository is addressable, versioned and immutable: anybody with permission can ask it for version 1.1.0 by name and get the same bytes, for as long as it exists. You will use **GitHub Packages**, which is an npm registry attached to your repository.

8.1 Move the project into the team repository, through a pull request

You protected `main` in Lab 04, so this is the first real test of that rule:

```
cd ~/devops/devops-project-TEAMNAME
git switch -c add-build-tool
cp -r ~/devops/lab05/{tools,src,test,Makefile,package.json,package-lock.json} .
printf 'node_modules/\nout/\n*.tgz\n' >> .gitignore
git add -A
git commit -m "Add npm build and packaging"
git push -u origin add-build-tool
```

> **Three things must not be in that commit.** `node_modules/`, `out/` and `*.tgz` are all **outputs**. Run `git status` after adding the `.gitignore` lines and before committing. If `git status` lists anything under `node_modules/`, stop and fix the ignore file — a repository with `node_modules` committed is a repository nobody can review.

Open the pull request, have a teammate review it, and merge. One member does this; the others pull.

8.2 Point the package at the repository

GitHub Packages requires a **scoped** name — the scope is the account that owns the repository. Edit `package.json`:

```
"name": "@rehannsarwar1/devops-pipeline",
"repository": {
  "type": "git",
  "url": "git+https://github.com/rehannsarwar1/devops-project-TEAMNAME.git"
},
"publishConfig": { "registry": "https://npm.pkg.github.com" },
```

Then tell npm where that scope lives, in a `.npmrc` **in the project**:

```
printf '@rehannsarwar1:registry=https://npm.pkg.github.com\n' > .npmrc
cat .npmrc
```

That file holds a registry address and nothing else. It is safe to commit, and it should be — every teammate needs it.

8.3 Authenticate — and keep the token out of the repository

You need a personal access token with the `write:packages` scope: GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token, tick `write:packages` (which selects `repo` with it). The token goes in your **home** `.npmrc`, never the project one:

```
npm config set //npm.pkg.github.com/:_authToken YOUR_TOKEN --location=user
grep -c authToken ~/.npmrc
```

> **This is the secrets rule from Week 04, and it is the one students actually break.** A token is a password. `~/.npmrc` is outside the repository, so Git cannot see it. A token in the project `.npmrc` is one `git add -A` away from being public forever — and deleting it in a later commit does **not** remove it from history. If you ever commit one: delete it on GitHub immediately and generate a new one. Rotating is the fix; editing the file is not.

Check before you publish, not after:

```
git check-ignore -v .npmrc || git status --short .npmrc
```

8.4 Publish

```
npm publish --dry-run
```

Expected output

```
npm notice package: @rehannsarwar1/devops-pipeline@1.1.0
npm notice filename: rehannsarwar1-devops-pipeline-1.1.0.tgz
npm notice total files: 3
npm notice Publishing to https://npm.pkg.github.com with tag latest (dry-run)
```

Read the registry line. If it says `registry.npmjs.org` your `publishConfig` has not taken effect and you are about to publish to the public internet. Fix it before continuing. Then:

```
npm publish
```

The package appears under the repository's **Packages** section on GitHub. That URL is what you submit.

8.5 Prove it is immutable

```
npm publish
```

The registry refuses it. The exact wording varies between registry versions, but it is a **409 Conflict** and it says the version already exists:

Expected output

```
npm error code E409
npm error 409 Conflict - PUT https://npm.pkg.github.com/@rehannsarwar1%2fdevops-pipeline
npm error Cannot publish over existing version
```

> **That refusal is the property the whole lecture was about.** Version 1.1.0 means one specific set of bytes, permanently. If you need to change something, you publish 1.1.1 — you do not amend 1.1.0, because somewhere there is a deployment that says it is running 1.1.0 and that statement has to keep being true. Immutability is not a restriction on you, it is a guarantee to everybody downstream.

Make a trivial change, then do it properly:

```
npm version patch
make ci
npm publish
```

Two versions now exist side by side in the repository. Neither has replaced the other.

8.6 Install it back, as a teammate would

```
cd /tmp && rm -rf team && mkdir team && cd team && npm init -y >/dev/null
printf '@rehannsarwar1:registry=https://npm.pkg.github.com\n' > .npmrc
npm install @rehannsarwar1/devops-pipeline@1.1.0
npx devops-build ~/devops/lab05/src/notes.md out.html && head -4 out.html
```

You asked for 1.1.0 by name and got it, even though 1.1.1 exists. That is what a repository is for.

> **If GitHub Packages will not authenticate.** It is a token-scope problem nine times out of ten — regenerate with `write:packages` ticked. If you still cannot publish, fall back to a **GitHub Release**: on the repository, Releases → Draft a new release → choose your existing `v1.1.0` tag → attach the `.tgz` as a binary → Publish. Submit that URL instead and say in your report that you used the fallback. A release asset is versioned and immutable too; what it is not is installable by name, which is the difference worth writing a sentence about.

8.7 Checkpoint

- The build tool is on the team repository's `main`, having arrived through a pull request.
- `node_modules/`, `out/` and `*.tgz` are not in the repository.
- Your token is in `~/.npmrc` and not in the project.
- At least one version is published and visible at a URL.
- You have seen the publish refused for an existing version.

9. Submission

Submit a zip named `Lab05_<your-roll-number>.zip` containing:

1. Your `.tgz` artifact.
2. `package.json`, `package-lock.json` and the final `Makefile`.
3. `tools/`, `src/` and `test/` — but **not** `node_modules/` or `out/`.
4. A `Lab05_report.md` with the answers below.

```
cd ~/devops/lab05
make clean
zip -r Lab05_21-NTU-CS-1234.zip . -x 'node_modules/*' -x '.git/*'
```

9.1 The report

- How many packages did you declare, and how many were installed? Paste your `npm ls --all` output.
- Point at the `deduped` line in it and explain in two sentences what it means and what would have happened without it.
- What does `^15.0.3` permit and forbid? Why is that a problem for a build that must be repeatable?
- Two things `package-lock.json` records that `package.json` does not. Paste your two identical hashes from 6.3.
- Paste the `npm ci` refusal from 6.4 and say in one sentence why that refusal is desirable.
- Which version number did you choose in 7.1, and why that one rather than the two you rejected?
- Paste the `tar -tzf` listing of your artifact and justify every file in it.
- Paste the `npm ls --all` output from 7.5 (the broken artifact) and explain the failure from the **tree**, not from the error message.
- The URL of your published package or release.
- If you used the `fallback/` Lab 02 files, or the GitHub Release fallback in 8.6, say so.

> **Three lines of real output beat a page of prose.** Every question above can be answered with a short paste and two sentences. A report that describes what the commands do, without showing that they ran on your machine, scores badly — `make clean` before you zip and the marker can tell the difference.

10. Marking rubric

Task 4 carries the marks. The other tasks are assessed through the report and the submitted files.

Criterion	Weight	What full marks looks like
Dependency comprehension	15 %	Direct and transitive counts from their own run; the `deduped` line identified and explained as a resolved diamond, not described as a duplicate.
Build delegation	15 %	Makefile delegates the recipe and keeps incremental behaviour; `node_modules` is a real target on `package-lock.json`, order-only on the pages; `make ci` passes from clean.
Lock file and reproducibility	15 %	Two identical hashes; names the transitive versions **and** the integrity checksums as things only the lock file records; explains the `npm ci` refusal as a safety property.
Versioning judgement	15 %	Correct semantic version with a reason that refers to the effect on a consumer, not to the size of the change. Tag present and matching.
Artifact correctness	25 %	Artifact contains only intended files, installs into an

Criterion	Weight	What full marks looks like
		empty directory and runs there. The `devDependencies` failure diagnosed from the dependency tree.
Publishing	15 %	A published, addressable version at a URL; token in `~/.npmrc` and nothing secret in the repository; immutability demonstrated and explained as a downstream guarantee.

Three things that cost marks every year. Committing `node_modules` — it is an output, and it is instantly visible in the zip. Answering the versioning question with "it was a big change" instead of saying what a consumer must do. And claiming the artifact works having only ever run it inside the directory that built it; Section 7.4 exists precisely because that proves nothing.

11. Troubleshooting

Symptom	Cause and fix
`node: command not found`	Node is not installed in WSL2 — installing it on Windows does not help. `sudo apt install -y nodejs npm` inside Ubuntu.
`usage: build.js <source.md> <dest.html>`	You left out the bare `--`. `npm run build -- src/x.md out/x.html`.
`Cannot find module 'markdown-it'`	Either `npm ci` has not run, or the dependency is under `devDependencies`. Section 7.5 is this, deliberately.
`npm ci` says the files are not in sync	You edited `package.json` by hand. Run `npm install` once to regenerate the lock file, then commit both together — always both.
`make: Nothing to be done for 'build'` when something did change	Make compares timestamps to the second. Wait a second and rerun. See 5.4.
`make: *** No rule to make target` for a `.html` file	Your pattern rule still says `.txt`, or SOURCES still globs `*.txt`.
`npm ERR! Invalid version`	The version must be `X.Y.Z` — three numbers, no `v` prefix. `npm version` writes it correctly; hand-editing often does not.
`npm version` fails with `Git working directory not clean`	Commit or stash first. The tag has to point at a commit that contains the change.
`node_modules` appears in `git status`	Your `.gitignore` is missing `node_modules/`, or the directory was committed before you ignored it. `git rm -r --cached node_modules` then commit.
`npm publish` → `401 Unauthorized`	No token, or the token lacks `write:packages`. Regenerate it with that scope ticked.
`npm publish` → `403 Forbidden`	The package scope does not match the account that owns the repository. `@owner/name`, where `owner` is exactly the GitHub account or organisation.
`npm publish` → `404` on the scope	The `.npmrc` registry line is missing or misspelt. `@owner:registry=https://npm.pkg.github.com`.
`npm publish` → `409 Cannot publish over existing version`	Expected, and the point of 8.5. Bump the version with `npm version patch`.
The artifact is suspiciously large	Your `files` field is missing or wrong, so npm packed everything it was allowed to. `npm pack --dry-run` lists what it would include.

12. Looking ahead

You now have a build that a machine other than yours could run: a manifest, a lock file, one entry point, and an artifact with a name that identifies it. Lab 06 points a build server at `make ci` so that every pull request is built, tested and packaged automatically — at which point the "require status checks to pass" setting you deliberately left alone in Lab 04 becomes something there are status checks for, and `main` starts being protected by evidence rather than by policy.

> **Keep the artifact and the tag.** Lab 06 reproduces this build on a server and compares the result against the artifact you made by hand today. Do not delete the `.tgz`, and do not delete the tag.